

Списки и кортежи

Уверенно работать с последовательностями: создавать списки и кортежи, обращаться по индексу, делать срезы и изменять список методами.

DAY 11

LIST

tuple

index

slice

mutable

ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

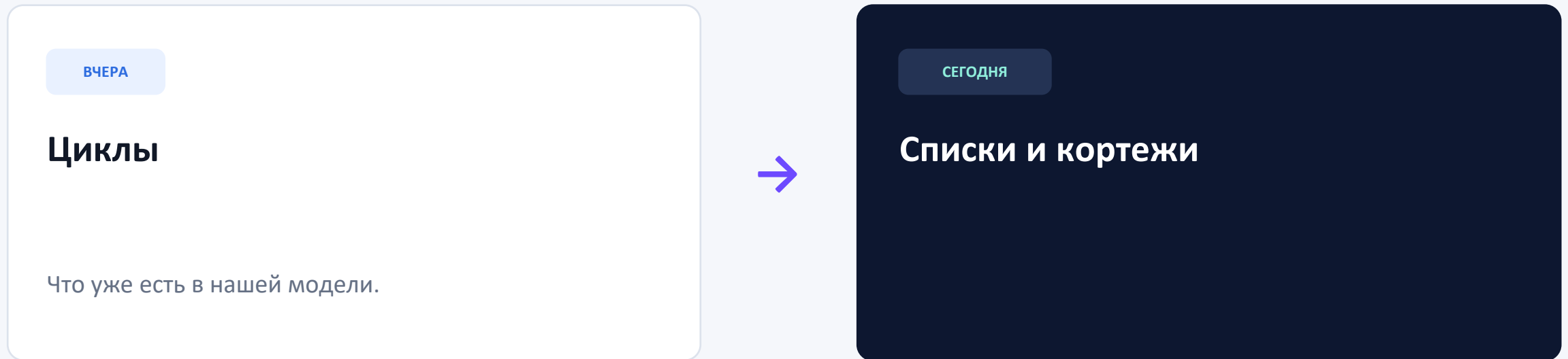
Конкретные действия вместо списка терминов.

- 01 Уверенно работать с последовательностями: создавать списки и кортежи, обращаться по индексу, делать срезы и изменять список методами.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.



Вчера: Циклы. Сегодня мы добавляем следующий слой: списки и кортежи.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Коллекции позволяют хранить не одно значение, а целую группу связанных данных и обрабатывать её единым способом.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

Список — редактируемый список покупок; кортеж — зафиксированная запись, которую не планируем менять.

ПЕРЕВОД В IT

list

изменяемая упорядоченная коллекция значений

tuple

неизменяемая упорядоченная коллекция

index

номер позиции элемента, начинающийся с нуля

Слой 1: list • tuple • index

Сначала смысл, затем профессиональный термин.

**list**

изменяемая упорядоченная коллекция значений

**tuple**

неизменяемая упорядоченная коллекция

**index**

номер позиции элемента, начинающийся с нуля

Слой 2: slice • mutable • method

Сначала смысл, затем профессиональный термин.

**slice**

выбранный диапазон элементов последовательности

**mutable**

способность объекта изменяться после создания

**method**

функция, принадлежащая объекту, например `list.append`

Слой 3: element • append

Сначала смысл, затем профессиональный термин.

element

отдельное значение внутри коллекции

append

добавление элемента в конец списка

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: length • comprehension

Сначала смысл, затем профессиональный термин.

**length**

количество элементов коллекции

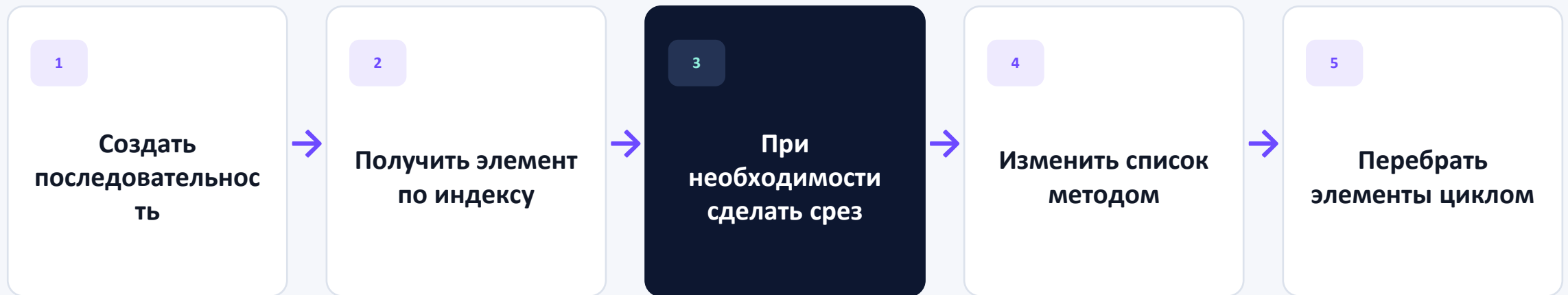
**comprehension**

компактная запись создания новой коллекции

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day11_step1

```
skills = ["Python", "Git", "SQL"]  
skills.append("Flask")  
print(skills)  
print(skills[0])  
print(skills[1:3])
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
skills = ["Python", "Git", "SQL"]
```

Сохраняем значение или результат вычисления под понятным именем.

```
skills.append("Flask")
```

Python выполняет эту строку сверху вниз.

```
print(skills)
```

Показываем результат, чтобы увидеть состояние программы.

```
print(skills[0])
```

Показываем результат, чтобы увидеть состояние программы.

```
print(skills[1:3])
```

Показываем результат, чтобы увидеть состояние программы.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



```
skills = ["Python", "Git", "SQL"]
skills.append("Flask")
print(skills)
print(skills[0])
print(skills[1:3])
```

до запуска

РЕЗУЛЬТАТ

```
['Python', 'Git', 'SQL', 'Flask']
Python
['Git', 'SQL']
```

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.



changed

```
point = (59.43, 24.75)
print(point[0])
print(point[1])
```



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

59.43
24.75

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.

```
items = ["a", "b", "c"]  
print(items[3])
```

wrong

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

Индексация начинается с нуля. У трёх элементов допустимы индексы 0, 1 и 2. Индекс 3 вызывает IndexError.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

Исправленный вариант

Меняем только то, что действительно было причиной.

```
items = ["a", "b", "c"]  
print(items[-1])
```

fixed

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Вести список оценок: добавить новую, заменить ошибочную и показать первые три значения.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **Создаём список оценок.**
- 2 Добавляем новый элемент через `append()`.
- 3 Меняем элемент по индексу.
- 4 Берём срез первых трёх элементов.
- 5 Печатаем итог.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.

```
grades = [4, 5, 3]  
grades.append(5)  
grades[2] = 4
```

solution_part1

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



```
grades = [4, 5, 3]
grades.append(5)
grades[2] = 4
print(grades)
print(grades[:3])
```

solution_final

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

```
[4, 5, 4, 5]  
[4, 5, 4]
```

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Собери список покупок, добавь хлеб, удали молоко и выведи количество оставшихся позиций.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

append() добавляет в конец, remove() удаляет конкретное значение, len() считает элементы.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
shopping = ["молоко", "яблоки"]  
shopping.append("хлеб")  
shopping.remove("молоко")  
print(shopping)  
print(len(shopping))
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

```
['яблоки', 'хлеб']  
2
```

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

МИНИ-ПРОЕКТ

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую законченную задачу.

ЗАДАНИЕ

Сохрани три координаты как кортежи внутри списка и выведи каждую точку.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



mini_project

```
points = [(1, 2), (3, 4), (5, 6)]  
for x, y in points:  
    print("x=", x, "y=", y)
```

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

x= 1 y= 2

x= 3 y= 4

x= 5 y= 6

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01 Забывать, что индексы начинаются с 0.

02 Менять список во время обхода без понимания последствий.

03 Путать `append()` и `extend()`.

04 Использовать кортеж там, где данные должны изменяться.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «list» и какую проблему это понятие помогает описать?

2

Чем «list» отличается от «slice»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 **list — изменяемая упорядоченная коллекция значений.**

2 list: изменяемая упорядоченная коллекция значений. slice: выбранный диапазон элементов последовательности.

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3

```
['Python', 'Git', 'SQL', 'Flask']  
Python  
['Git', 'SQL']
```

4

Индексация начинается с нуля. У трёх элементов допустимы индексы 0, 1 и 2. Индекс 3 вызывает `IndexError`.

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Уверенно работать с последовательностями: создавать списки и кортежи, обращаться по индексу, делать срезы и изменять список методами.
- ✓ Читать и объяснять пример темы «Списки и кортежи».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Словари и множества

Завтра: Словари и множества. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.